【扔物线 Compose 课程讲义】5.14-ModifierLocal 和 ModifierLocalProvider、ModifierLocalConsumer

关于讲义

讲义是对课程内容中**所有结论的集中汇总**。所以在看完每节课程之后，迅速来看一遍讲义，对课程内容做回顾，会有非常好的吸收效果。

面试前不要背讲义

在面试之前，通过快速浏览一遍讲义来确认所有知识都没有忘记，是个很好的备战方式。

但**不要用【背讲义上的知识点】的方式来备战**。最有效的浏览方式是：

浏览讲义中的所有内容，并：

- 看到「懂，并且记得背后的原因（或技术细节）」的，直接跳过；
- 看到「懂，但细节已经忘了」的，快速回看当节课程，把知识捡起来；
- 看到「已经不太懂 / 完全忘了」的，认真重看当节课程，把知识学回来。

疑问或意见

如果你发现讲义或课程有错误，或有别的什么意见，可以向任何一个你认识的助教反馈。

如果认可我们的课程，还请给个好评！ 评价链接：https://rengwuxian.com/cp

作用

用于让数据穿透 `Modifier`，实现在多个 `Modifier` 之间的共享。

写法

扔物线学堂 - 秉心 (1437182)

1. 直接用 `Modifier.modifierLocalProvider()` 和 `Modifier.ModifierLocalConsumer()`

Kotlin | 复制代码

```kotlin
Modifier
  .modifierLocalProvider(sharedKey,) { "rengwuxian" }
  .xxxxxx()
  .modifierLocalConsumer { println("共享字符串: ${sharedKey.current}") }
```

详情见课程视频。

2. 实现 `ModifierLocalProvider` 和 `ModifierLocalConsumer` 接口

扔物线学堂 - 秉心 (1437182)

详情还是看课程吧 Kotlin 复制代码

```kotlin
Modifier
  .then(object : LayoutModifier, ModifierLocalProvider<String> {
    val sharedString = "rengwuxian"
    override fun MeasureScope.measure(
      measurable: Measurable,
      constraints: Constraints
    ): MeasureResult {
      val placeable = measurable.measure(constraints)
      return layout(placeable.width, placeable.height) {
        placeable.placeRelative(0, 0)
      }
    }

    override val key: ProvidableModifierLocal<String>
    get() = sharedWidthKey
    override val value: String
    get() = sharedString
  })
  .then(object : LayoutModifier, ModifierLocalConsumer {
    lateinit var sharedString: String
    override fun MeasureScope.measure(
      measurable: Measurable,
      constraints: Constraints
    ): MeasureResult {
      val placeable = measurable.measure(constraints)
      sharedString.xxxx
      return layout(placeable.width, placeable.height) {
        placeable.placeRelative(0, 0)
      }
    }

    override fun onModifierLocalsUpdated(scope: ModifierLocalReadScope) = with(scope) {
      sharedString = sharedWidthKey.current
    }
  })
```

详情见课程视频。

原理

- 存储方面和别的大多数 `Modifier` 一样；
- 回调方面，是在 `NodeChain.updateFrom()` 里，保存完之后直接触发（所以早于测量和布局

过程）。